

Algoritmos

Alondra Ludueña

2026-05-13

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a MatLab que trabaja con variables que en general pueden verse como matrices.

```
a=38
A=40
a=40
b=60
c=b-a
c
```

```
## [1] 20
```

```
y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

Exploración de Datos Preinstalados R incluye una librería de bases de datos para práctica. Para ver el catálogo de opciones disponibles, se utiliza el comando:

- `data()`: Abre una lista de todos los datasets internos que puedes cargar.

Visualización y Acceso a la Información Cuando trabajamos con un dataset específico, como `cars` (datos sobre velocidad y distancia de frenado), tenemos dos formas principales de interactuar con él:

- Llamada Global (`cars`): Escribir el nombre del objeto muestra la estructura completa de la base de datos (filas y columnas). Es útil para entender la relación entre las variables.
- Acceso por Columna (`cars$speed`): El símbolo `$` actúa como un filtro o selector. Permite extraer una sola columna específica del dataset para trabajar con ella como un vector independiente.

```
summary(cars)
```

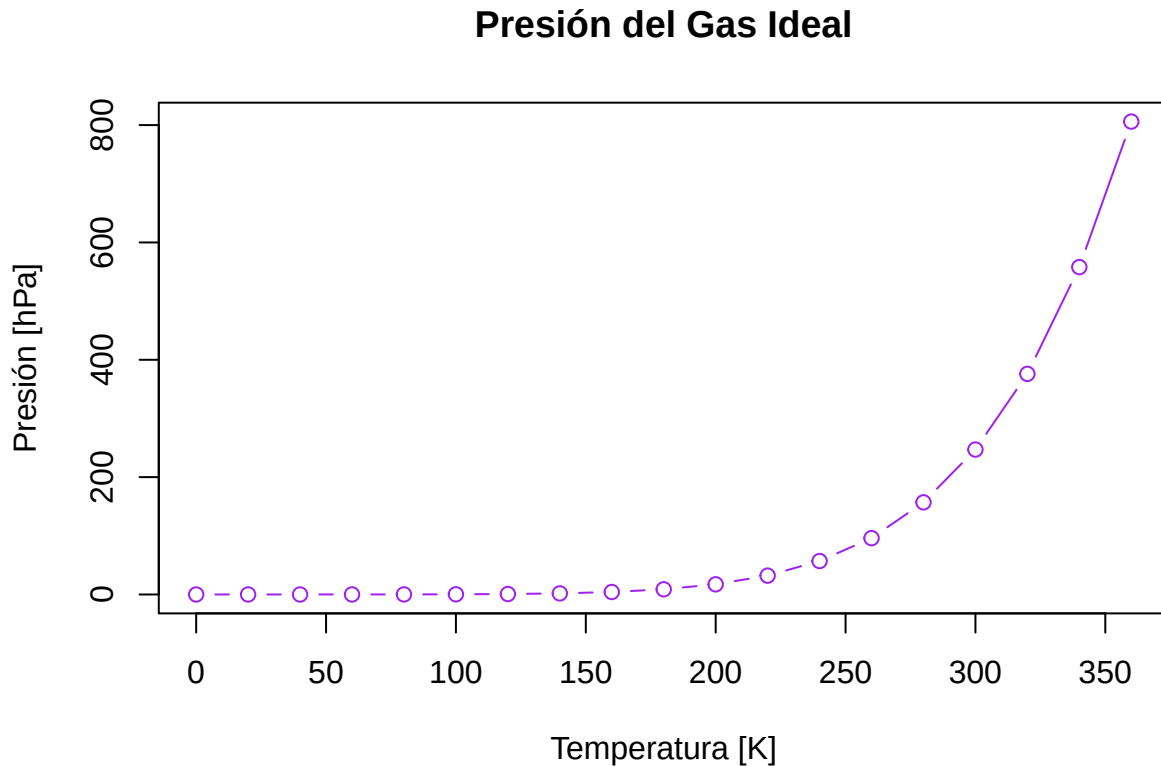
```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
```

```
## Mean :15.4 Mean : 42.98
## 3rd Qu.:19.0 3rd Qu.: 56.00
## Max. :25.0 Max. :120.00
```

Including Plots

You can also embed plots, for example:

```
plot(pressure,main="Presión del Gas Ideal",ylab="Presión [hPa]",xlab="Temperatura [K]",type="b", col="purple")
```



Comando plot

Es un comando que permite graficar cualquier fuente de datos que le asignamos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al gráfico. En caso de duda de como utilizar el comando podemos escribir `##?plot##` en la consola, lo cual abrirá una ventana que explicará a detalle el comando.

Funciones estadísticas

```
z1<-rnorm(350,21,5)
z1
```

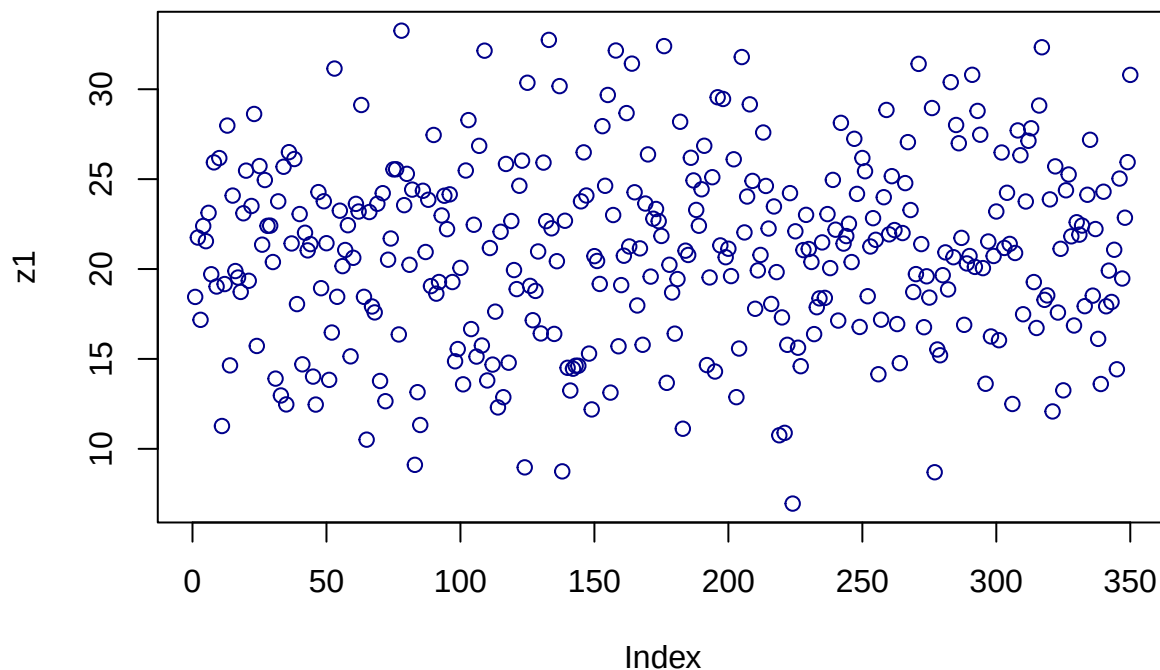
```
## [1] 18.448952 21.751072 17.175327 22.394093 21.548300 23.124899 19.710176
## [8] 25.931202 19.027579 26.170479 11.269024 19.163511 27.978815 14.640440
## [15] 24.084868 19.887938 19.526967 18.725997 23.100158 25.472498 19.346283
## [22] 23.506310 28.624078 15.721838 25.727082 21.355282 24.949522 22.402630
## [29] 22.413996 20.389248 13.904323 23.763824 12.966887 25.689156 12.469870
## [36] 26.499474 21.422201 26.117941 18.049739 23.056748 14.699079 22.018096
## [43] 21.036887 21.386483 14.017374 12.462180 24.277662 18.934432 23.769733
## [50] 21.436199 13.835694 16.467961 31.149040 18.455105 23.245241 20.161618
```

```

## [57] 21.058384 22.437479 15.143205 20.624658 23.623898 23.198162 29.123351
## [64] 18.454232 10.509763 23.169951 17.916267 17.587651 23.629776 13.774816
## [71] 24.218484 12.647890 20.518264 21.697448 25.543237 25.549694 16.360141
## [78] 33.256680 23.553441 25.293543 20.236137 24.416730 9.110123 13.149098
## [85] 11.325194 24.352409 20.946162 23.852633 19.051558 27.456879 18.632004
## [92] 19.278162 22.978048 24.078486 22.215657 24.152765 19.276493 14.868807
## [99] 15.549965 20.064305 13.587243 25.479355 28.283098 16.660700 22.462070
## [106] 15.131896 26.851749 15.746916 32.144637 13.802960 21.167868 14.684280
## [113] 17.629650 12.302744 22.072651 12.871034 25.844147 14.789539 22.670062
## [120] 19.942298 18.883206 24.630763 26.026434 8.968588 30.360933 19.061356
## [127] 17.152798 18.783577 20.976968 16.416703 25.920101 22.663958 32.743943
## [134] 22.263509 16.390881 20.434625 30.173485 8.745858 22.684389 14.506490
## [141] 13.242210 14.457027 14.623255 14.618972 23.764316 26.487760 24.089869
## [148] 15.296800 12.192268 20.716467 20.446457 19.170334 27.945762 24.629214
## [155] 29.681407 13.126836 23.004032 32.153318 15.703916 19.106514 20.740282
## [162] 28.672442 21.254732 31.424263 24.269285 17.973705 21.148839 15.787036
## [169] 23.642569 26.374712 19.579834 22.797933 23.338451 22.661844 21.839288
## [176] 32.398011 13.669650 20.238546 18.693444 16.405093 19.440981 28.190761
## [183] 11.117160 21.020313 20.781503 26.188002 24.927403 23.286823 22.407404
## [190] 24.434528 26.856270 14.657546 19.531996 25.107461 14.299128 29.553015
## [197] 21.313021 29.459301 20.658059 21.129490 19.607097 26.103089 12.873328
## [204] 15.578423 31.789887 22.034986 24.033353 29.158812 24.900767 17.789775
## [211] 19.923039 20.783946 27.592233 24.621226 22.251736 18.057474 23.479391
## [218] 19.828459 10.753370 17.308195 10.887287 15.784881 24.222374 6.954983
## [225] 22.098070 15.623178 14.595695 21.053727 23.012143 21.113053 20.391925
## [232] 16.379396 17.876958 18.368115 21.480862 18.393612 23.055849 20.050699
## [239] 24.957828 22.188228 17.133963 28.129280 21.415265 21.833088 22.519036
## [246] 20.388179 27.243002 24.178901 16.777518 26.174934 25.439131 18.487536
## [253] 21.253853 22.829506 21.625420 14.148445 17.181912 23.995093 28.848271
## [260] 21.929334 25.164645 22.167706 16.939617 14.765421 22.000673 24.779544
## [267] 27.056806 23.280662 18.720326 19.717618 31.411152 21.387076 16.764327
## [274] 19.595554 18.412329 28.954419 8.697669 15.526380 15.204183 19.658173
## [281] 20.925671 18.866145 30.392621 20.658308 28.011761 26.994891 21.732233
## [288] 16.895871 20.311692 20.723126 30.802965 20.119731 28.793229 27.469165
## [295] 20.050856 13.618833 21.531086 16.249491 20.725547 23.200137 16.039941
## [302] 26.480859 21.173195 24.241532 21.396601 12.493646 20.891828 27.708349
## [309] 26.329583 17.485075 23.756363 27.131338 27.829441 19.275488 16.721749
## [316] 29.094266 32.335271 18.286578 18.534399 23.872804 12.079087 25.708860
## [323] 17.580246 21.124315 13.251948 24.379359 25.257666 21.819534 16.859260
## [330] 22.601639 21.912139 22.417999 17.933867 24.131060 27.188034 18.520490
## [337] 22.223678 16.114061 13.604707 24.300785 17.929075 19.911079 18.169503
## [344] 21.072006 14.424426 25.031704 19.472297 22.854788 25.944252 30.797693

```

```
plot(z1,col="darkblue") #comentario
```



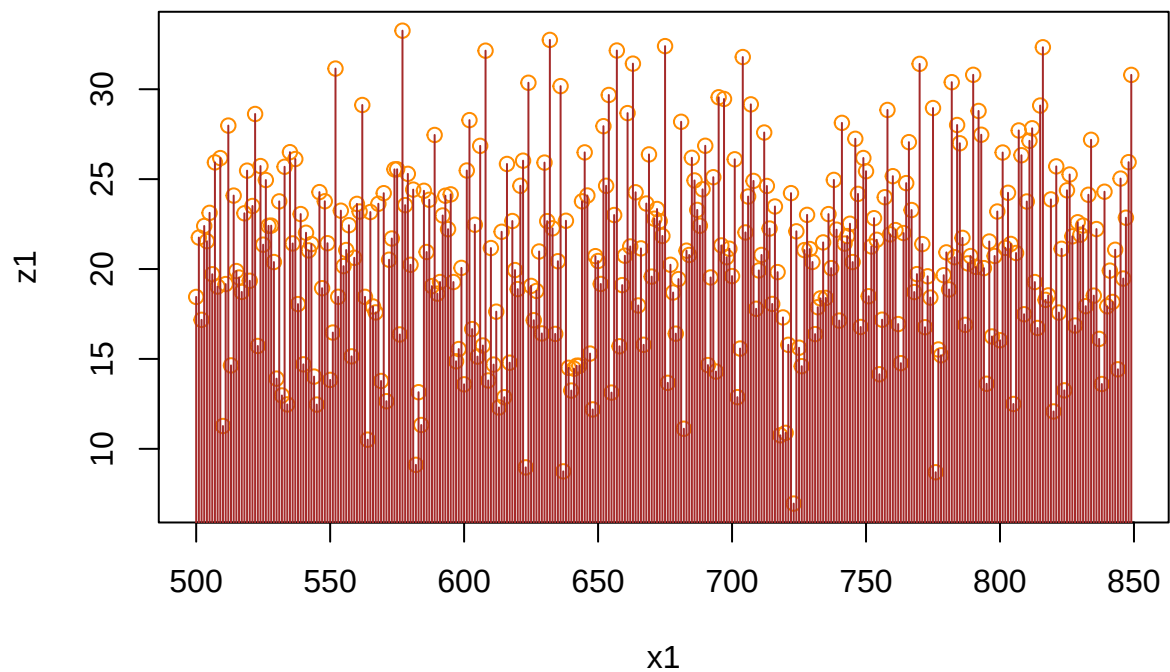
```
w1<-length(z1)
w1
```

```
## [1] 350
```

```
x1<-(500:849)
x1
```

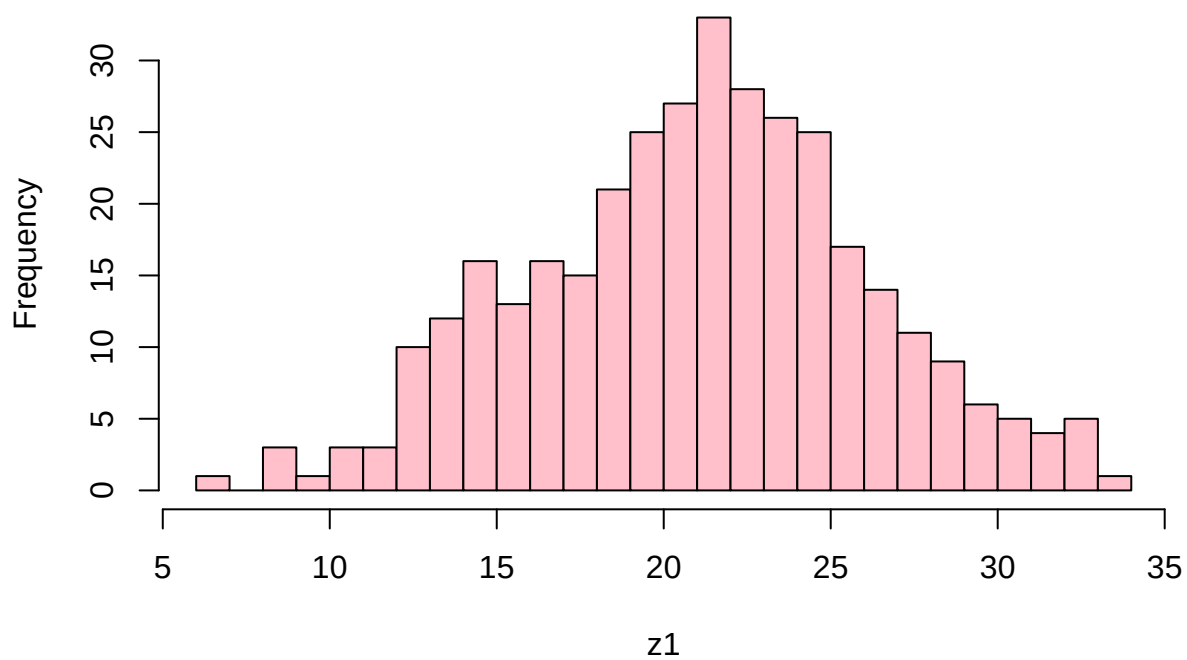
```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(x1,z1,col="darkorange")
lines(x1,z1,col="brown",type="h")
```



```
hist(z1, main="Histograma de edades",breaks=20,col="pink")
```

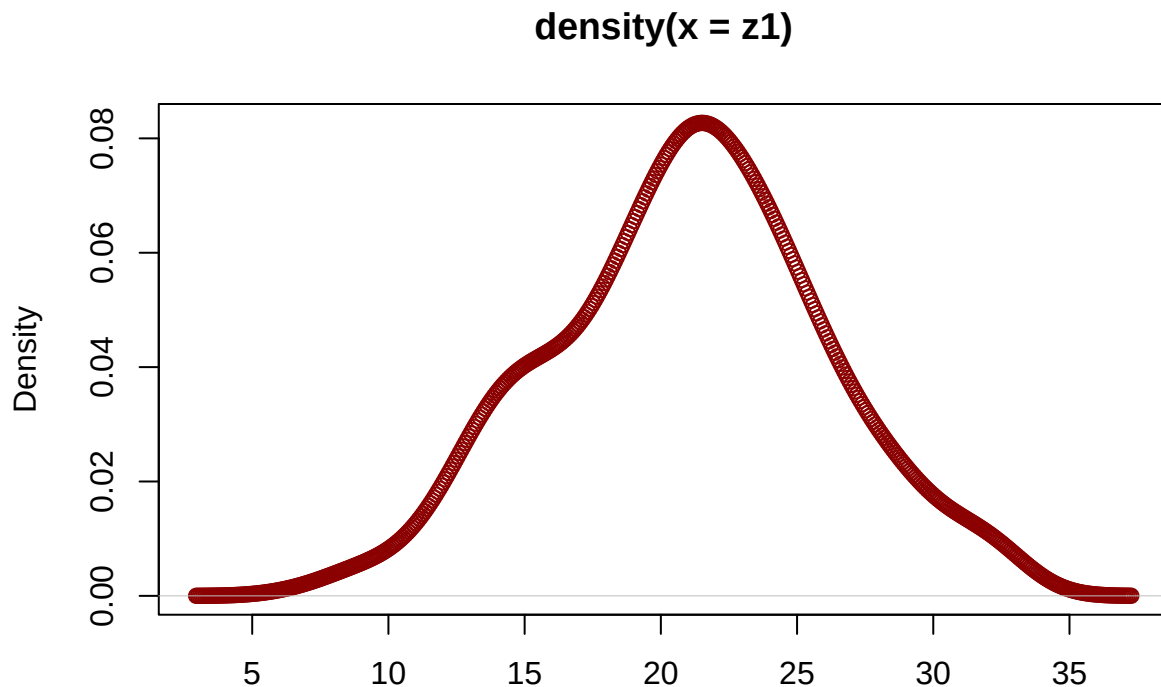
Histograma de edades



```
density(z1)
```

```
##
## Call:
## density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.337
```

```
##
##      x      y
## Min.   : 2.943   Min.   :9.739e-06
## 1st Qu.:11.525   1st Qu.:3.998e-03
## Median :20.106   Median :2.085e-02
## Mean   :20.106   Mean    :2.908e-02
## 3rd Qu.:28.687   3rd Qu.:4.872e-02
## Max.   :37.268   Max.    :8.270e-02
plot(density(z1),col="darkred",type="b")
```



N = 350 Bandwidth = 1.337

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Ejercicios de algoritmos

Las últimas 3 cifras de mi DNI son 848

Ejercicio 1 Crear una variable que tenga ese número

```
dni=848
```

Ejercicio 2 Crear un vector que tenga todos los números desde el 1 hasta el 848 enteros

```
secuencia_dni<-(1:dni)
secuencia_dni
```

```
##      [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
##     [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
##     [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
##     [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
##     [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
```

```

## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848

```

Ejercicio 3 Calcular la suma de todos los valores del vector `secuencia_dni`

```
sum(1:length(secuencia_dni))
```

```
## [1] 359976
```

Ejercicio 4 Repetir el ejercicio anterior en Python

```
secuencia_dni=range(1,848)
total=sum(secuencia_dni)
print(total)
```

```
## 359128
```

Ejercicio 5 Repetir el ejercicio anterior con Julia. No se programó para no ocupar memoria.

Ejercicio 6 Determinar CPU time used.

```
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##      user  system elapsed
## 0.016 0.000 0.017
```

```
require(stats)
system.time(for(i in 1:length(sequencia_dni)){})
```

```
##      user  system elapsed
## 0.003 0.000 0.002
```

```
total<-0
valorfinal<-length(sequencia_dni)
for (i in 1:valorfinal){
total<-total+sequencia_dni[i]
}
print(total)
```

```
## [1] 359976
```

```
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

```
## Time difference of 14.01501 secs
```

Ejercicio 7 ¿Cuánto tarda en correr el código del ejercicio 2? con system.time

```
tiempo <- system.time({
  sum(1:length(sequencia_dni))
})
print(tiempo)
```

```
##      user  system elapsed
##      0      0      0
```

Biblioteca tictoc

Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando `install.packages()` o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones `tic` y `toc` son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluacion comparativa que el tiempo de sistema recién demostrado. Sin embargo, `tictoc` agrega mucha más comodidad al usuario y armonía al conjunto.

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
```

```
## [1] "dormire una siestita..."
```



```
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
```

```
## [1] "...suena el despertador"
## [1] "...suena el despertador"
toc()
```

```
## sleeping: 2.005 sec elapsed
```

Biblioteca rbenchmark

La documentación de la función benchmark del paquete rbenchmark R lo describe como un simple contenedor alrededor de system.time. Sin embargo, agrega mucha conveniencia en comparación con las llamadas simples a system.time. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

Implementación de una serie Fibonacci o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales: 0, 1, 1, 2, 3, 5, 8 ... 89, 144, 233 ... La sucesión comienza con los números 0 y 1, 2 a partir de estos, cada término es la suma de los dos anteriores, es la relación de recurrencia que la define. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca Universidad de Florencia. Liber Abachi- Autor Fibonacci

Ejercicio 8 ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```
# Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2    # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
  siguiente <- a + b
  a <- b
  b <- siguiente
  serie <- c(serie, b) # agregamos a la serie
  contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
```

```
## Cantidad de iteraciones: 9
```

```
cat("Primer número mayor a 20:", b, "\n")
```

```
## Primer número mayor a 20: 21
```

```
cat("Serie de Fibonacci hasta ese número:\n")
```

```
## Serie de Fibonacci hasta ese número:
```

```
print(serie)
```

```
## [1] 0 1 1 2 3 5 8 13 21
```

Ejercicio 9 Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```
#### Instalar paquete
if (!require(microbenchmark)) {
  install.packages("microbenchmark")
  library(microbenchmark)
}

## Loading required package: microbenchmark

#### Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
#### Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

#### Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
  burbuja = bubble_sort(vector),
  sort_R = sort(vector),
  times = 5
)
print(resultado)
```

```
## Unit: microseconds
##      expr      min       lq      mean     median      uq
## burbuja 23027104.05 23872652.26 2.396741e+07 24180707.038 24259831.441
## sort_R    764.14    829.57 9.725286e+02    949.311    1107.111
##      max neval
## 24496750.091    5
##    1212.511    5
```

La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto impropio lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado

previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

Ejercicio 10 Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×1010 y verifica cuál de los dos es más eficiente.

```
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
  suma1 <- suma1 + i
}
suma1

## [1] 510555

#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2

## [1] 510555

#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({
  suma1 <- 0
  for (i in 1:1010){
    suma1 <- suma1 + i
  }
})

##      user      system elapsed
##    0.002    0.000    0.002

# Método 2
system.time({
  suma2 <- 1010 * (1010 + 1) / 2
})

##      user      system elapsed
##         0         0         0
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1×1010 . El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente

##Matriz Inversa y Gráficos Violín En este apartado se realiza una comparación del rendimiento computacional de distintos métodos para estimar los coeficientes de un modelo de regresión lineal en R. Para ello, se emplea la librería *microbenchmark*, la cual permite medir con precisión los tiempos de ejecución de diferentes algoritmos a través de múltiples repeticiones.

Se genera un conjunto de datos simulado utilizando funciones de generación aleatoria, compuesto por una matriz de variables explicativas X de dimensión 10000×100 y un vector respuesta y, construido como una combinación lineal de X con un término de error. Esto permite trabajar en un contexto controlado pero representativo de problemas de gran escala.

A continuación, se implementan tres enfoques para el cálculo de los coeficientes: el método clásico mediante la función `lm()`, el cálculo explícito usando la inversa de la matriz y la resolución directa del sistema lineal equivalente mediante la función `solve()`. Para asegurar la validez de la comparación, se define una función que verifica que los coeficientes obtenidos por los distintos métodos sean numéricamente equivalentes dentro de una tolerancia establecida.

Finalmente, se visualizan los resultados utilizando la librería `ggplot2`, a través de la función `autoplot()`, que permite representar gráficamente la distribución de los tiempos de ejecución de cada método, facilitando su análisis comparativo.

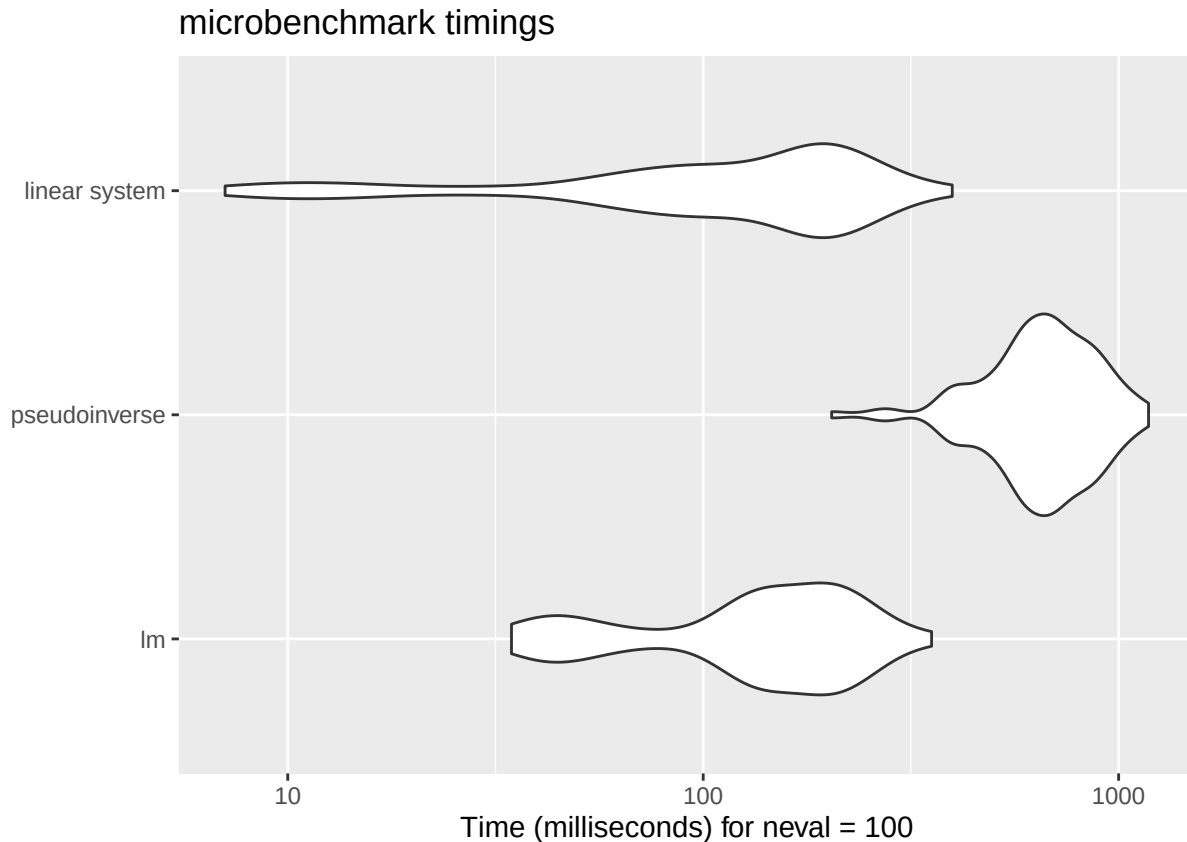
```
library(microbenchmark)
set.seed(2017)
n <- 10000
p <- 100
X <- matrix(rnorm(n * p), n, p)
y <- X %*% rnorm(p) + rnorm(100)
check_for_equal_coefs <- function(values) {
  tol <- 1e-12
  max_error <- max(
    c(
      abs(values[[1]] - values[[2]]),
      abs(values[[2]] - values[[3]]),
      abs(values[[1]] - values[[3]])
    )
  )
  max_error < tol
}
mbm <- microbenchmark(
  "lm" = {
    b <- lm(y ~ X + 0)$coef
  },
  "pseudoinverse" = {
    b <- solve(t(X) %*% X) %*% t(X) %*% y
  },
  "linear system" = {
    b <- solve(t(X) %*% X, t(X) %*% y)
  },
  check = check_for_equal_coefs
)
mbm

## Unit: milliseconds
##          expr          min          lq         mean        median          uq        max neval
##          lm    34.558938   90.22286  148.8261  138.9076  208.8432  354.6586   100
## pseudoinverse 203.834594 566.12542  676.4588  677.5187  801.8564 1180.3594   100
## linear system   7.065116  69.35385  136.4068  135.6027  198.3918  397.6217   100

library(ggplot2)
autoplot(mbm)

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()``.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
## Please report the issue at
## <https://github.com/joshuaulrich/microbenchmark/issues/>.
```

```
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



La

línea: `autoplot(mbm)` genera automáticamente un gráfico a partir del objeto de microbenchmark, y por defecto ese gráfico es un boxplot (diagrama de cajas), que muestra: mediana, cuartiles, dispersión de los tiempos. No es un gráfico de violín. Entonces, este gráfico que sigue es mucho mejor que solo boxplot. El violín deja ver si hay: colas largas, multimodalidad y variabilidad real.

```
library(ggplot2)
autoplot(mbm) +
  geom_violin(fill = "skyblue", alpha = 0.4, color = NA) + # distribución
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.7) + # resumen
  stat_summary(fun = mean, geom = "point",
    shape = 23, size = 3, fill = "red") + # media
  labs(
    title = "Comparación de tiempos de ejecución",
    x = "Método",
    y = "Tiempo (ms)"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold")
  )
```

Comparación de tiempos de ejecución

